10: Caches

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University

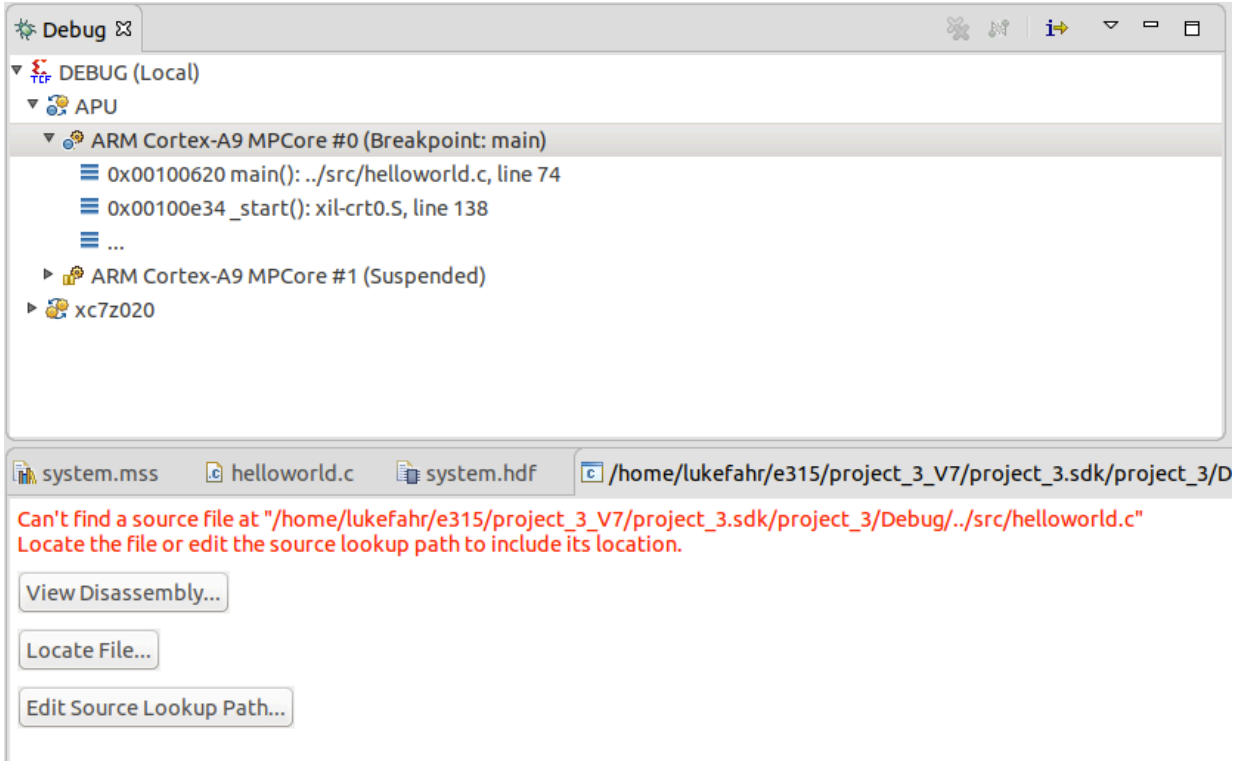
Some material taken from EECS370 at U. of Michigan

Let’s talk P3

- I had almost given up fixing P3.
- But I think I have it now*.

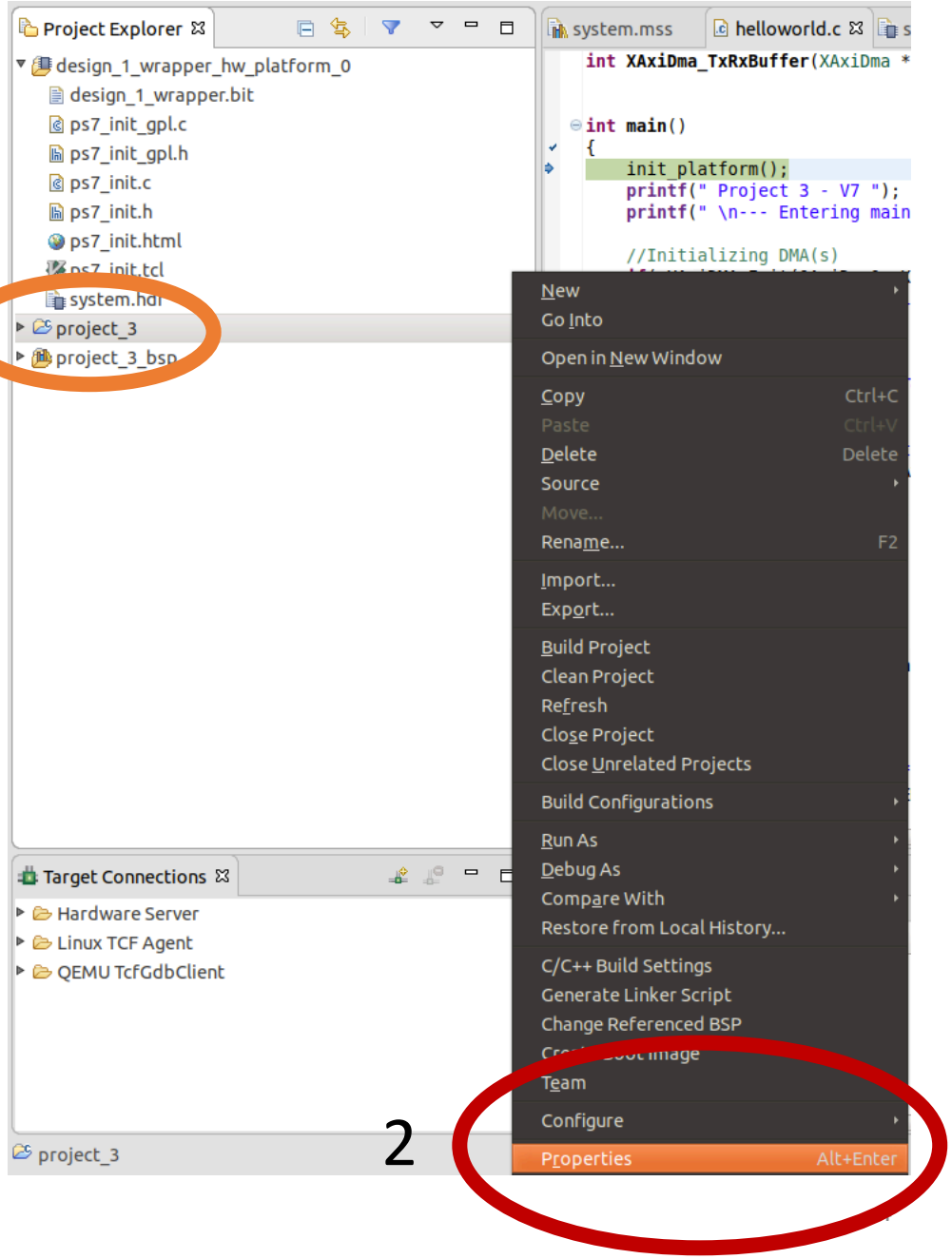
- *You get to help.

Step 0: Fail

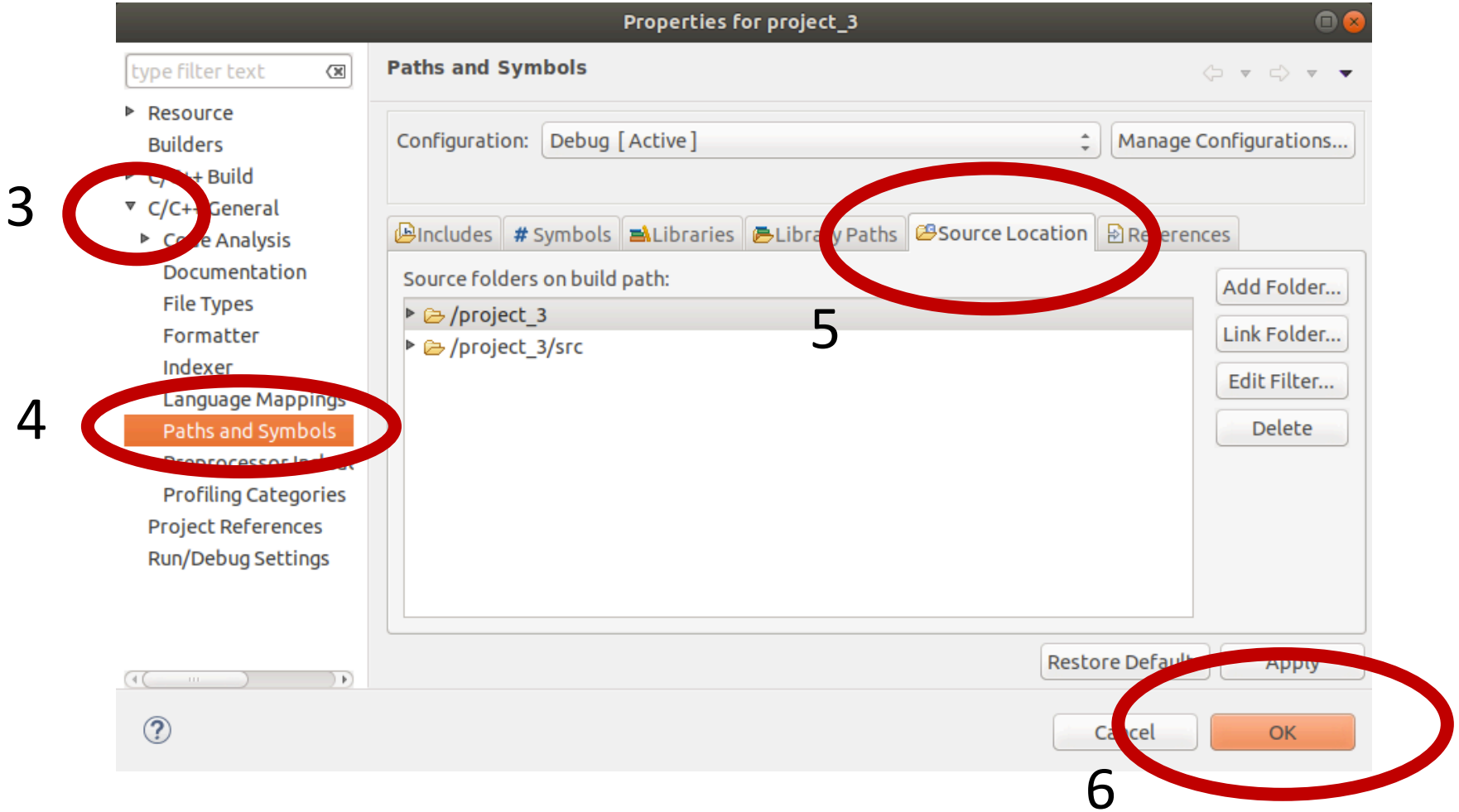

Debug
DEBUG (Local)
APU
ARM Cortex-A9 MPCore #0 (Breakpoint: main)
0x00100620 main(): ../src/helloworld.c, line 74
0x00100e34 _start(): xil-crt0.S, line 138
...
ARM Cortex-A9 MPCore #1 (Suspended)
xc7z020
system.mss
helloworld.c
system.hdf
/home/lukefahr/e315/project_3_V7/project_3.sdk/project_3/D
Can't find a source file at "/home/lukefahr/e315/project_3_V7/project_3.sdk/project_3/Debug/../src/helloworld.c"
Locate the file or edit the source lookup path to include its location.
View Disassembly...
Locate File...
Edit Source Lookup Path...

Step 1: Edit Properties

1 (Right)



Step 2: “OK” on Source Location



Now it works!

Debug

- DEBUG (Local)
  - APU
    - ARM Cortex-A9 MPCore #0 (Breakpoint: main)
      - 0x00100620 main(): ../src/helloworld.c, line 74
      - 0x00100e34 _start(): xil-crt0.S, line 138
      - ...
    - ARM Cortex-A9 MPCore #1 (Suspended)
  - xc7z020

system.mss | helloworld.c | system.hdf

```
int XAxiDma_TxRxBuffer(XAxiDma * axiDma, uint16_t DeviceId, uint32_t * TxBufferPtr, uint32_t * RxBu

int main()
{
    init_platform();
    printf(" Project 3 - V7 ");
    printf(" \n--- Entering main() ---  \n");
```

Why does that fix it?

- Don’t Ask. I have no idea.
- But it does.

More magic incantations to make things work!

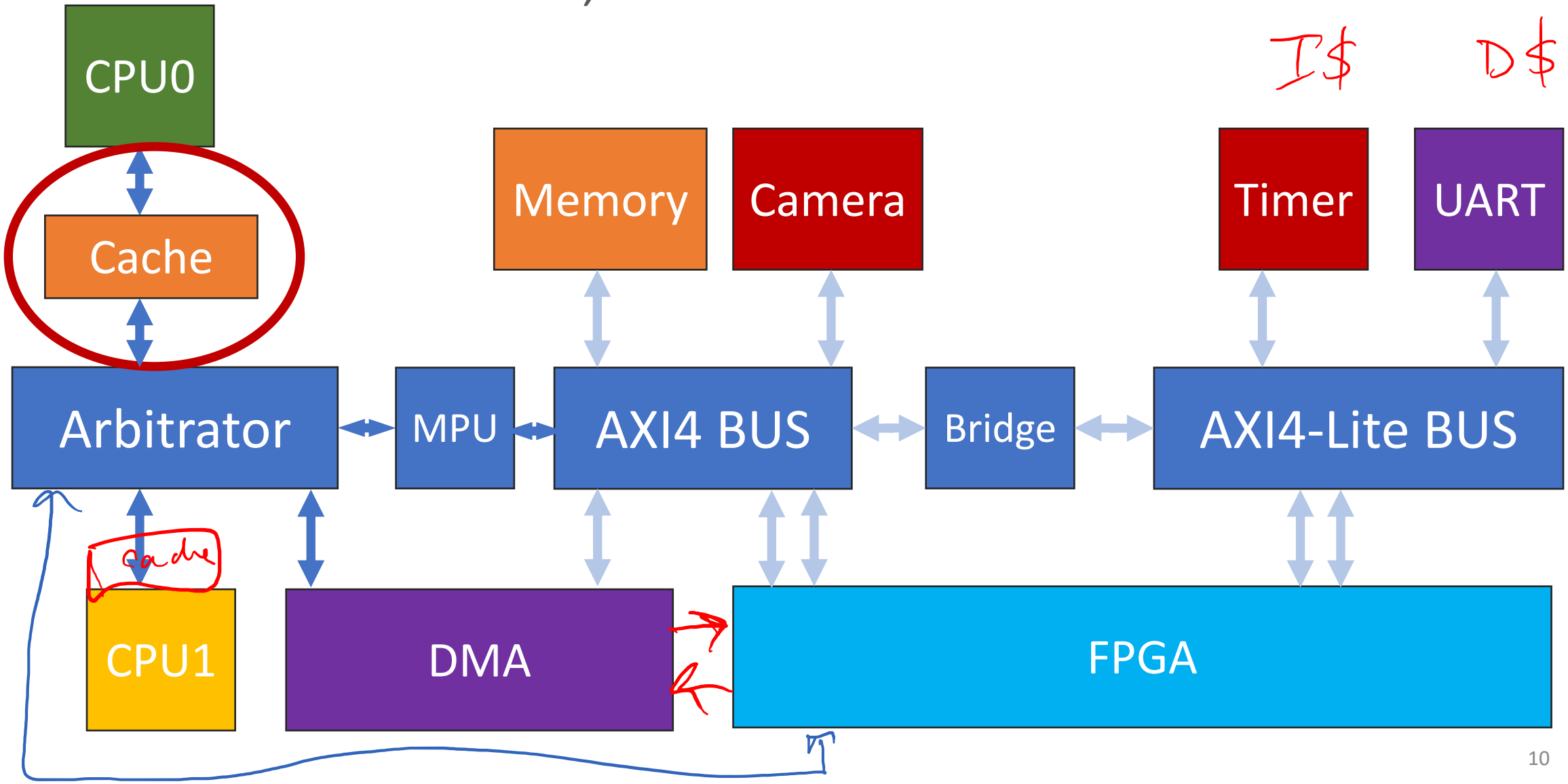
```
Xil_DCacheFlushRange((UINTPTR)RxBufferPtr,
BufferSizeBytes);
```

What is “DCache”?

Machine Model, V4

Cache = $

I$ D$



Memory

- Two structures that hold data:
  - Register file (little array of words).
  - Memory (bigger array of words).
- Several methods of implementing storage devices:
  - Static memory (made with logic gates).
  - Dynamic memory (transistor and capacitor).
  - ROM, and other ROM-like storage, e.g., flash (floating gate transistors).

Memory Hierarchy

- We want LOTS of memory
  - ARMv7 (or x86) can handle 2^{32} bytes of memory.
  - ARMv8 (or x86-64) can handle 2^{64} bytes of memory.

- What are our choices?
  - SRAM, DRAM, ROM, disk, tape, DVD?

Option 1: SRAM

- Fast: ~2ns access time or faster
  - Decoders are big
  - Array is big
    - Why?

- Expensive, high area requirement.
  - SRAM: $5.0 per megabyte
    - $20,000 for ARMv7
    - $88,000,000,000,000 for x86-64.

Option 2: DRAM

- Slower: ~60ns access time
  - Hurry up and wait design philosophy doesn't work
  - Stall for dozens of cycles on each memory load

- Less expensive than SRAM
  - DRAM costs $0.012 per megabyte
    - $50 for ARMv7
    - $210,000,000,000 for x86-64

Option 3: Flash

- Slower still: ~250ns access time
  - Hurry up and wait design philosophy doesn't work
  - Stall for hundreds of cycles on each memory load
- Less expensive than SRAM
  - Flash costs $0.0012 per megabyte
    - $4.9 for ARMv7
    - $21,000,000,000 for x86-64
- Non-Volatile

Option 4: Disks

- Obnoxiously slow: 3,000,000ns access time
  - We could have stopped with the Intel 4004
- Cheap
  - Disk storage costs $0.000043 per megabyte
    - $0.18 for ARMv7
    - $760,000,000 for x86-64
- Non-volatile

Option 5: Optical Disks

- Even slower: 100,000,000,000ns access time
  - Depends mostly on speed of finding and loading media

- DVD-R/DVD+R media costs about $0.00025 per megabyte.
  - $1.0 for ARMv7
  - $4,400,000,000 for x86-64

- Non-volatile.

Memory Hierarchy Goals

- **Fast:** Ideally run at processor clock speed
  - 1 ns access
- **Cheap:** Ideally free
  - Not more expensive than rest of system

- Options DRAM, flash, disks, and optical disks are too slow
- Option SRAM is too expensive

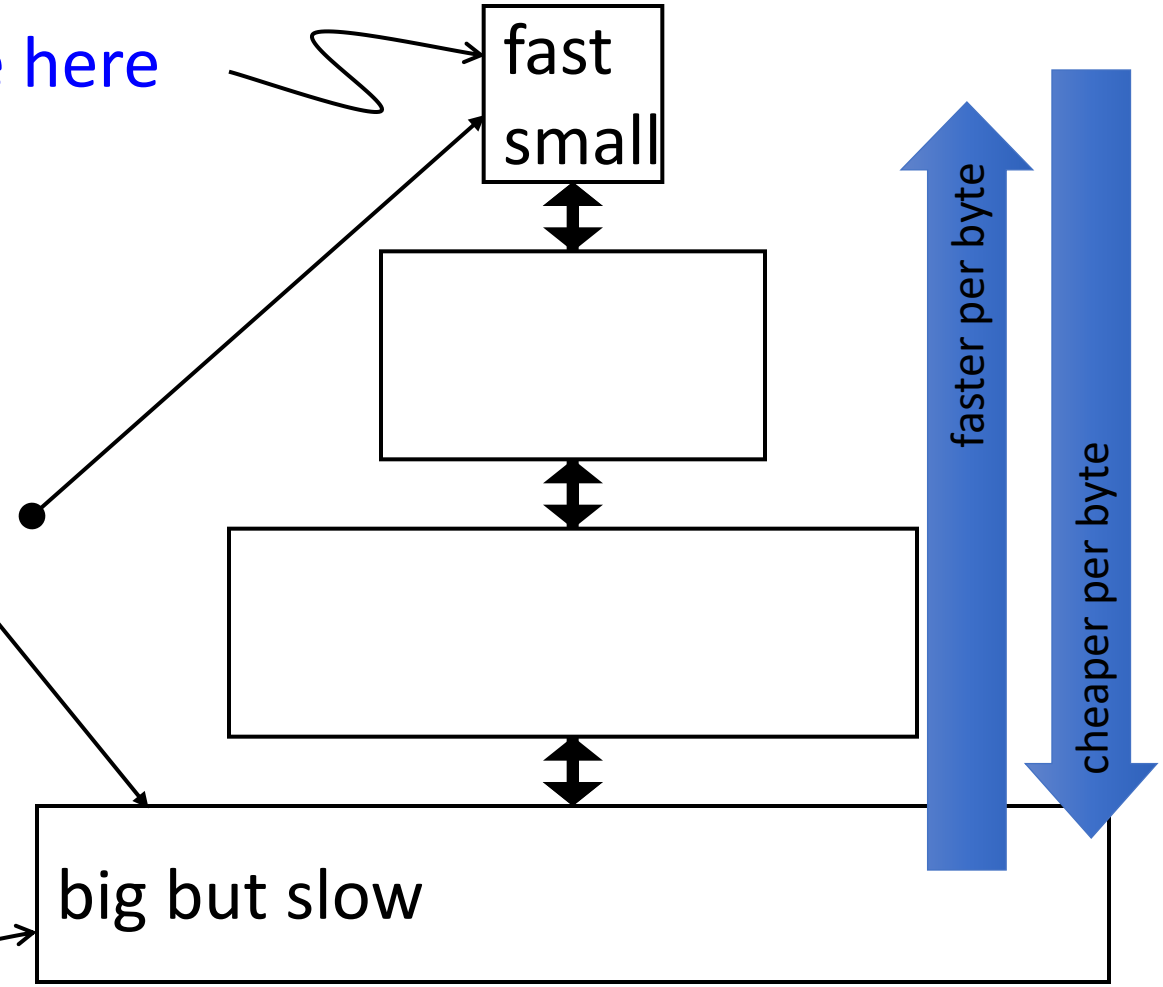
- Combine multiple memory technologies?

Memory Hierarchy Goals

move what you use here

With good locality of reference, memory appears as fast as and as large as

backup everything here



Memory Hierarchy

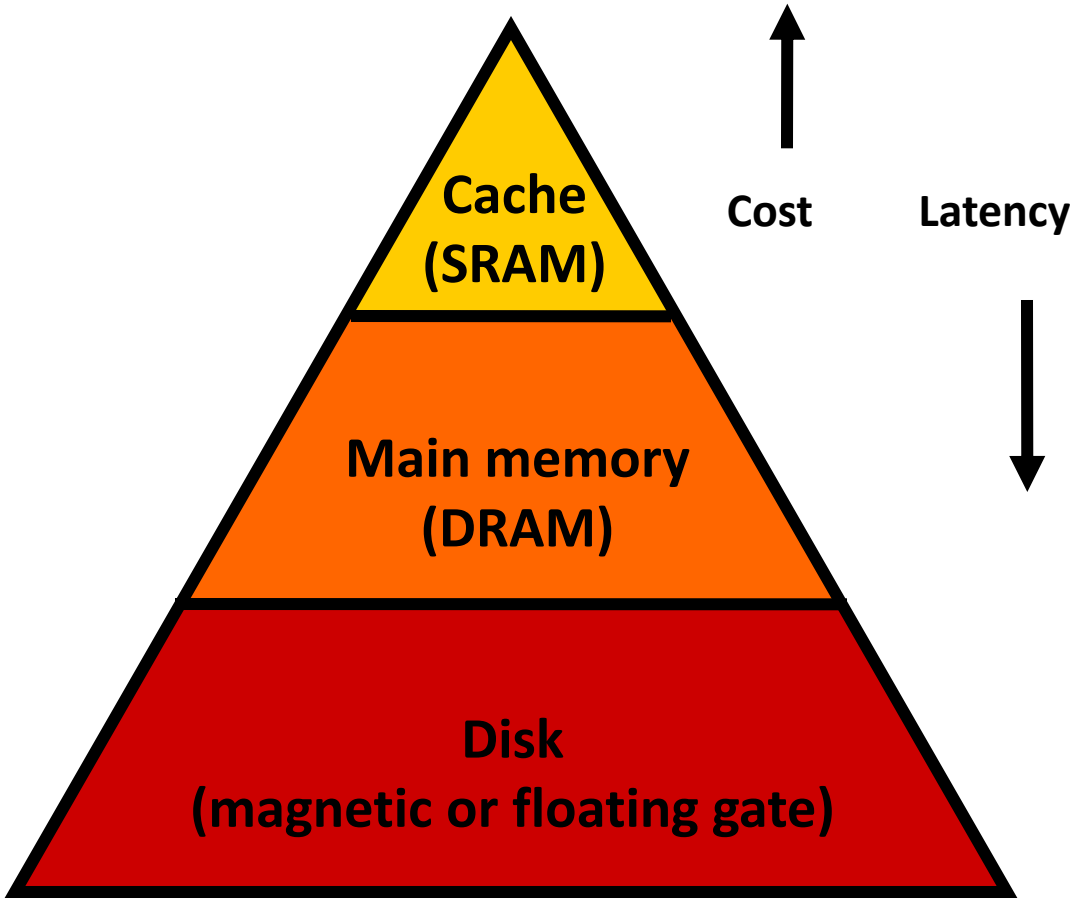
- Use a small array of SRAM
  - Fast!!!
  - For the **cache** (hopefully covers most loads and stores)

- Use a larger amount of DRAM
  - Cheaper than SRAM, faster than flash/disk
  - For the **main memory**

- Use a lot of flash and/or disk
  - Non-volatile. Cheap. Big.

- Don't try to buy 2^{64} bytes of anything
  - **Virtual Memory** makes it look like the entire address range is available
  - A few TB is enough for most machines today

Memory hierarchy



Definitions

- The **architectural** view of memory is
  - Defined by Instruction-Set Architecture (ISA)
  - What the programmer sees
  - Just a big array

- Breaking up the memory system into different pieces – cache, main memory and disk– is **not architectural**
  - The programmer doesn't know about it
  - A new implementation may not break it up in the same way
  - Part of the microarchitecture

Function of the Cache

- The cache will hold the data that we think is **most likely** to be **referenced**
  - Minimize the **average memory access latency**
  - Maximize the number of references that are serviced by the cache
  - How do we decide what the most likely accessed memory locations are?

Cache Analogy

- Studying books in library
  - **Option 1: Every time you switch to another book, return current book to shelf and get new book from shelf**
    - Latency = 5 minutes.
  - **Option 2: Keep 10 commonly-used books on shelf above desk**
    - Latency = 1 minute.
  - **Option 3: Keep three books open to appropriate locations on desk**
    - Latency = 10 seconds.

Example Problem

Given the following:
Cache has 1 cycle access time.
Main memory has 100 cycle access time.
Disk has 10,000 cycles access time.

What is the average access time for 100 memory references if 90% of the cache accesses are hits and 80% of the accesses to main memory are hits?

Example Problem

What is the average access time for 100 memory references if 90% of the cache accesses are hits and 80% of the accesses to main memory are hits?

Avg latency = cache access latency + miss rate $\times$ miss penalty
(accesses to cache and memory happen in series)

= 1 + (1 – 0.9) * (100 + (1 – 0.8) * 10000)
= 211 cycles/access or 21100 cycles for 100 accesses

idx = 0 1 2 3 4

mem = [a] [b] [c] [d] [e]

How to design a cache?

- Cache memory can copy data from any part of main memory.

- It has 2 parts:
  - The **TAG** (CAM) holds the memory address
  - The **BLOCK** (SRAM) holds the memory data

RAM: what is the value at address mem[4]

= e

addr	data
addr	data

TAG **BLOCK**

CAM: what is the index of the value c?

= 2

- Accessing the cache: compare reference address with the tag
  - If they match, get the data from the cache block
  - If they don't match, get the data from main memory

CAMs: content addressable memories

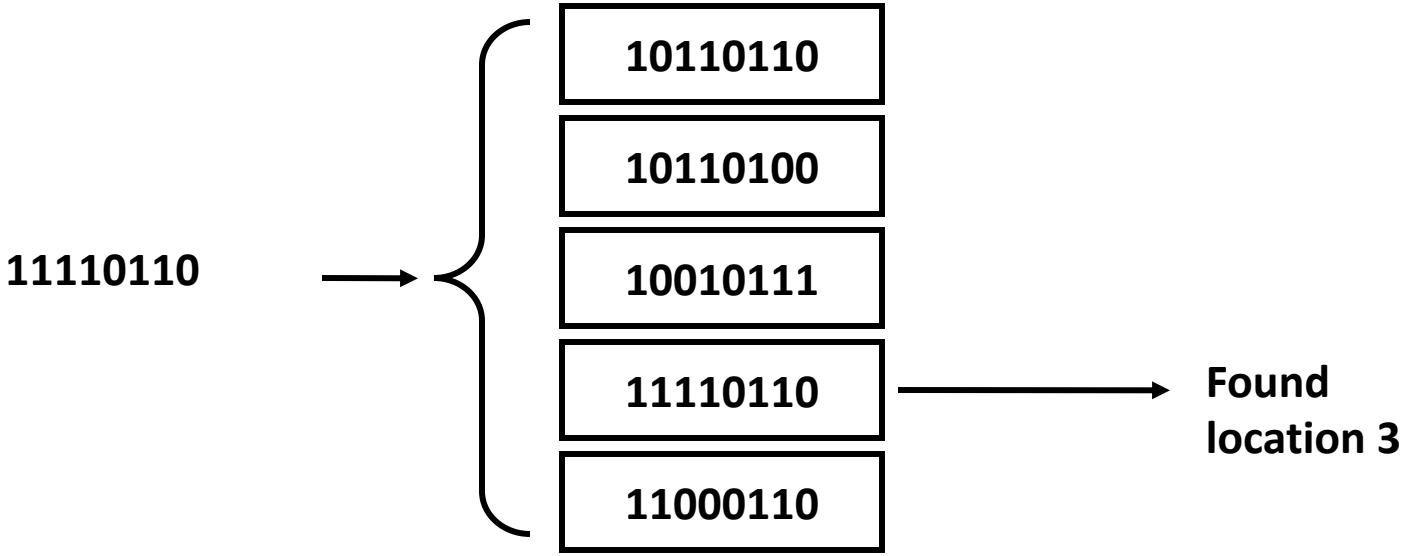
- Instead of thinking of memory as an array of data indexed by a memory address,
- Think of memory as a set of data matching a query
  - Instead of an address, we send a key to the memory, asking whether the key exists and, if so, what value it is associated with it
  - Memory answers: yes/no (hit/miss for caches) and gives associated value (if there is one)

Operations on CAMs

- **Search**: the primary way to access a CAM
  - Send data to CAM memory
  - Return “found” or “not found”
  - If found, return location of where it was found or associated value

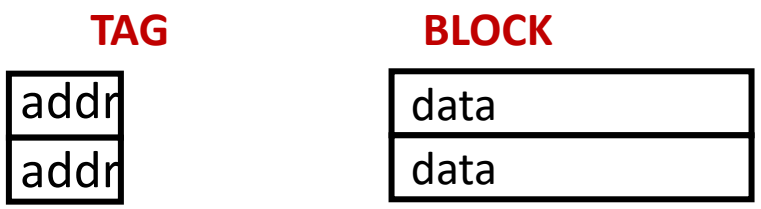
- **Write**:
  - Send data for CAM to remember
    - Where should it be stored if CAM is full?
      - Replacement policy
        - Replace oldest data in the CAM
        - Replace least recently searched data

CAM example



5 storage element CAM array of 8 bits each

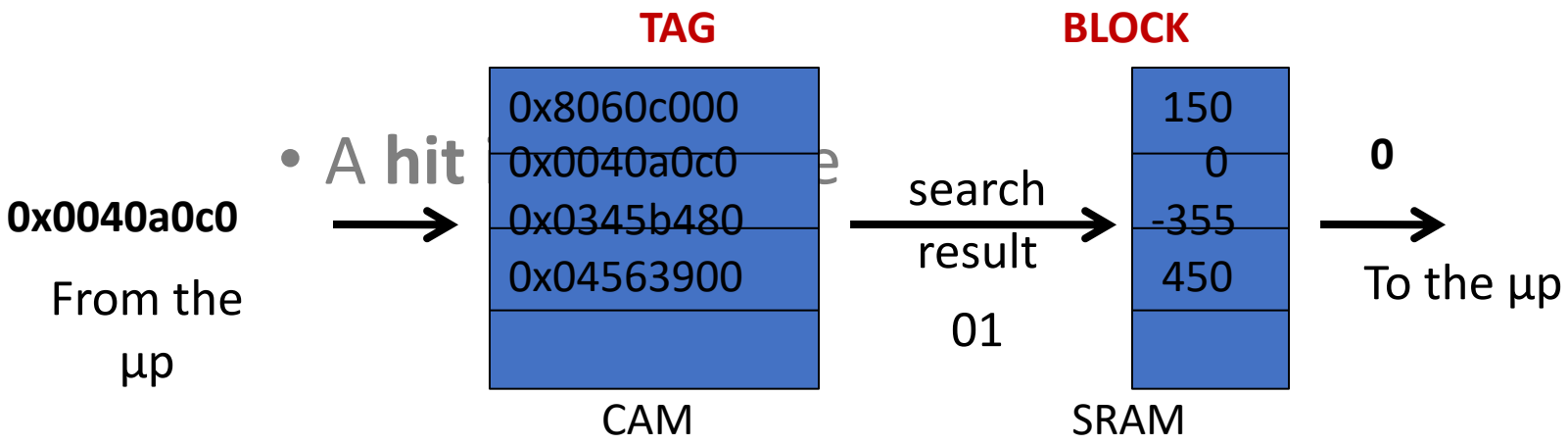
Cache Organization



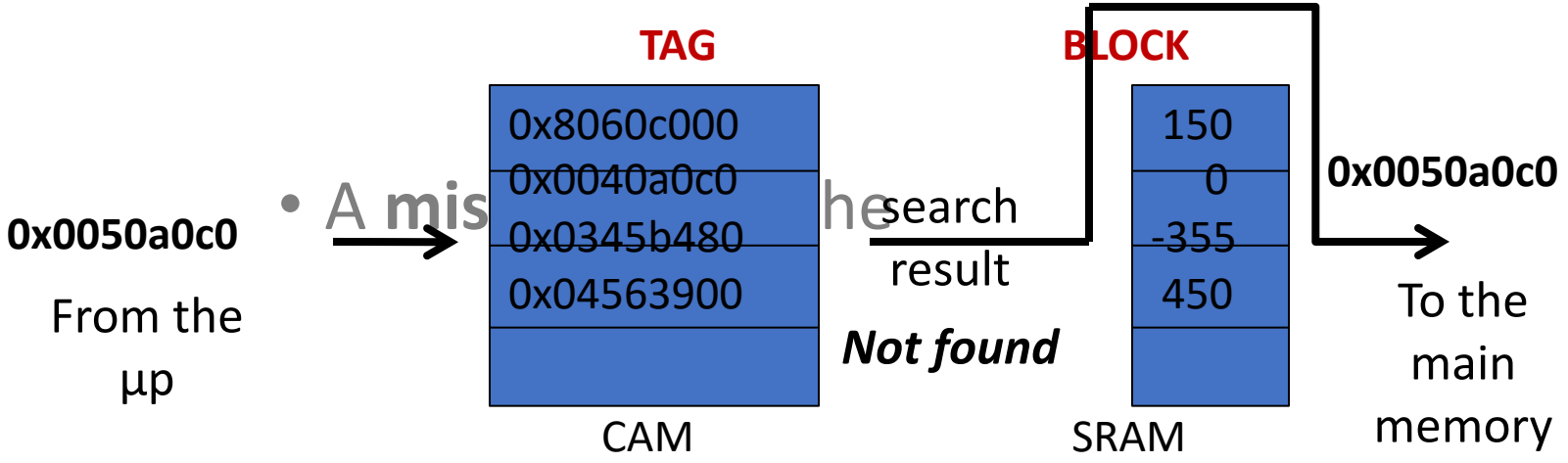
- A cache memory consists of multiple tag/block pairs (called cache lines)
  - Searches can be done in parallel (within reason)
  - At most one tag will match
- If there is a tag match, it is a cache HIT
- If there is no tag match, it is a cache MISS

Caches: the hardware view

- A **hit** [illegible]



- A **mis**[illegible]he



Cache Operation

- Every cache miss will get the data from memory and ALLOCATE a cache line to put the data in
  - Just like any CAM write
- Which line should be allocated?
  - Random? OK, but hard to grade test questions
  - Better than random? How?

Something To Think About

- Does an optimal replacement policy exist?
  - That is, given a choice of cache lines to replace, which one will result in the <u>fewest total misses during program execution</u>?
  - Hint: a crystal ball will come in handy in solving this problem…

- Why would we care?

Picking the Most Likely Addresses

- What is the probability of accessing a random memory location?
  - With no information, it is just as likely as any other address
- But programs are not random
  - They tend to use the same memory locations over and over
  - We can use this to pick the most referenced locations to put into the cache

Temporal Locality

- The principle of temporal locality in program references says:
- If you access a memory location (e.g., 0x1000)
- You will be more likely to re-access that location than some other random location
- What to do with a cache miss?
  - What if cache is full?

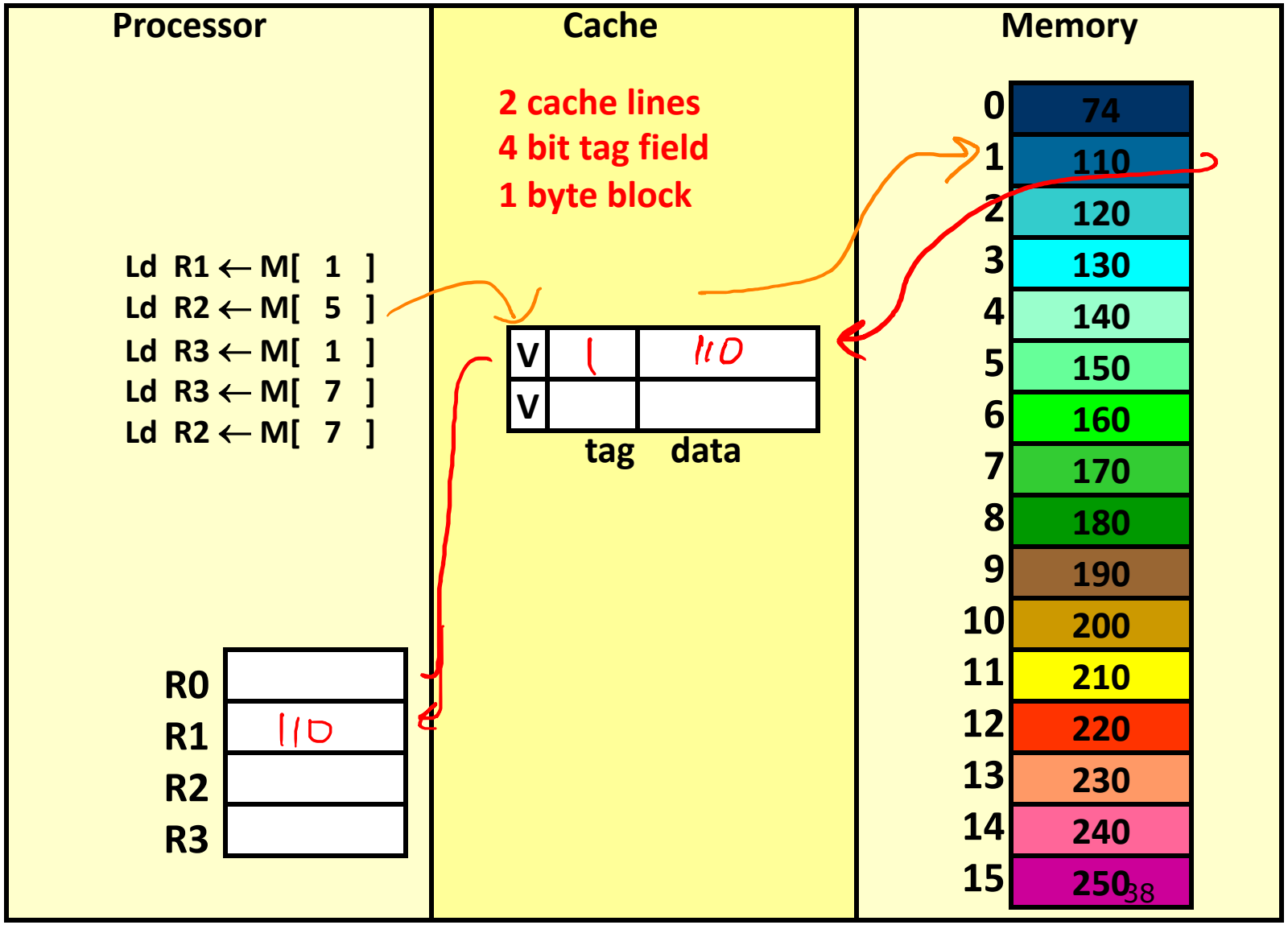
Which line to evict?

- Temporal locality says that the least recently referenced cache line should be evicted to make room for the new line.

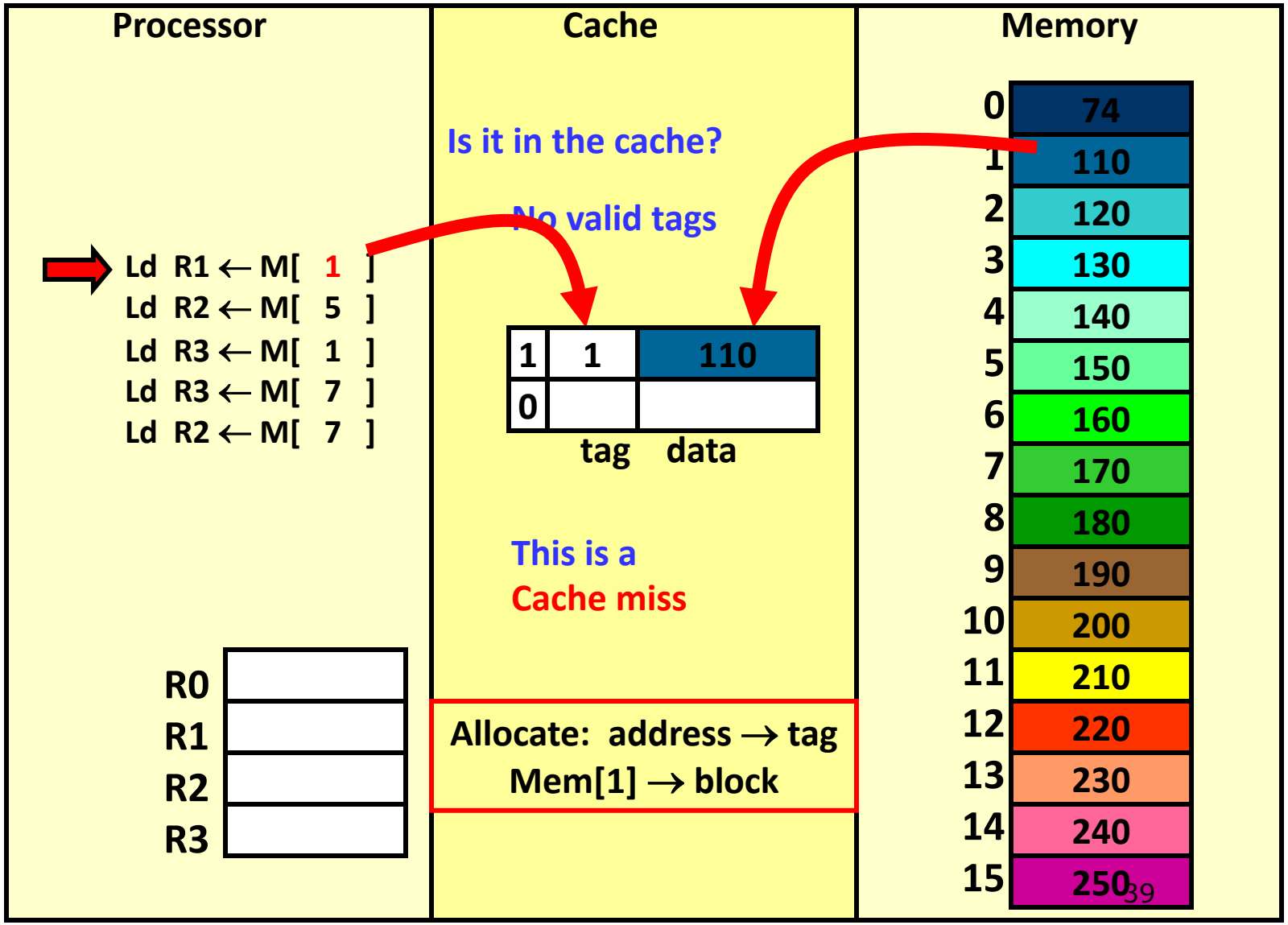
- Least Recently Used (LRU)

- Because the re-access probability falls over time as a cache line isn't referenced, the LRU line is least likely to be re-referenced

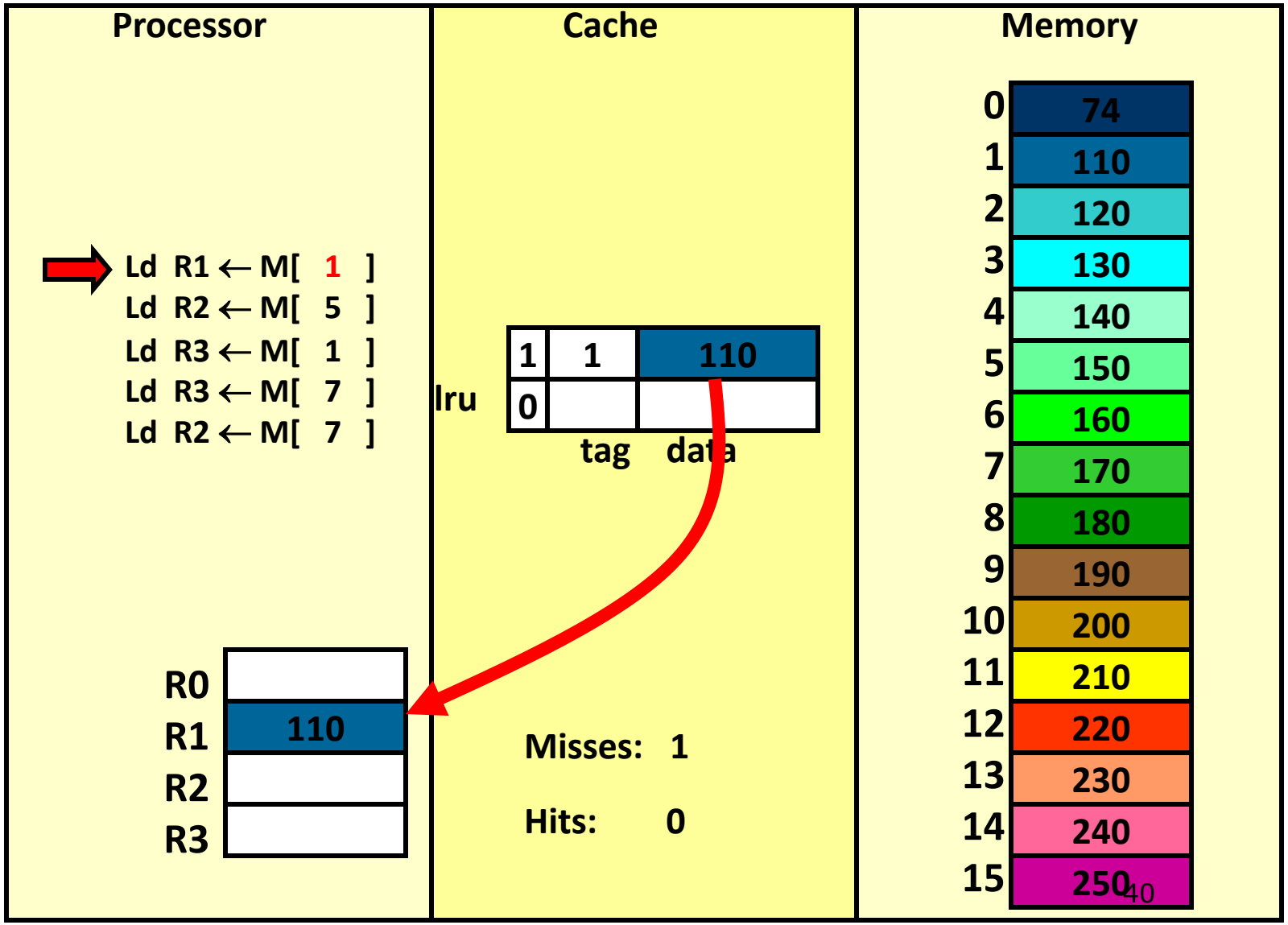
A Very Simple Memory System



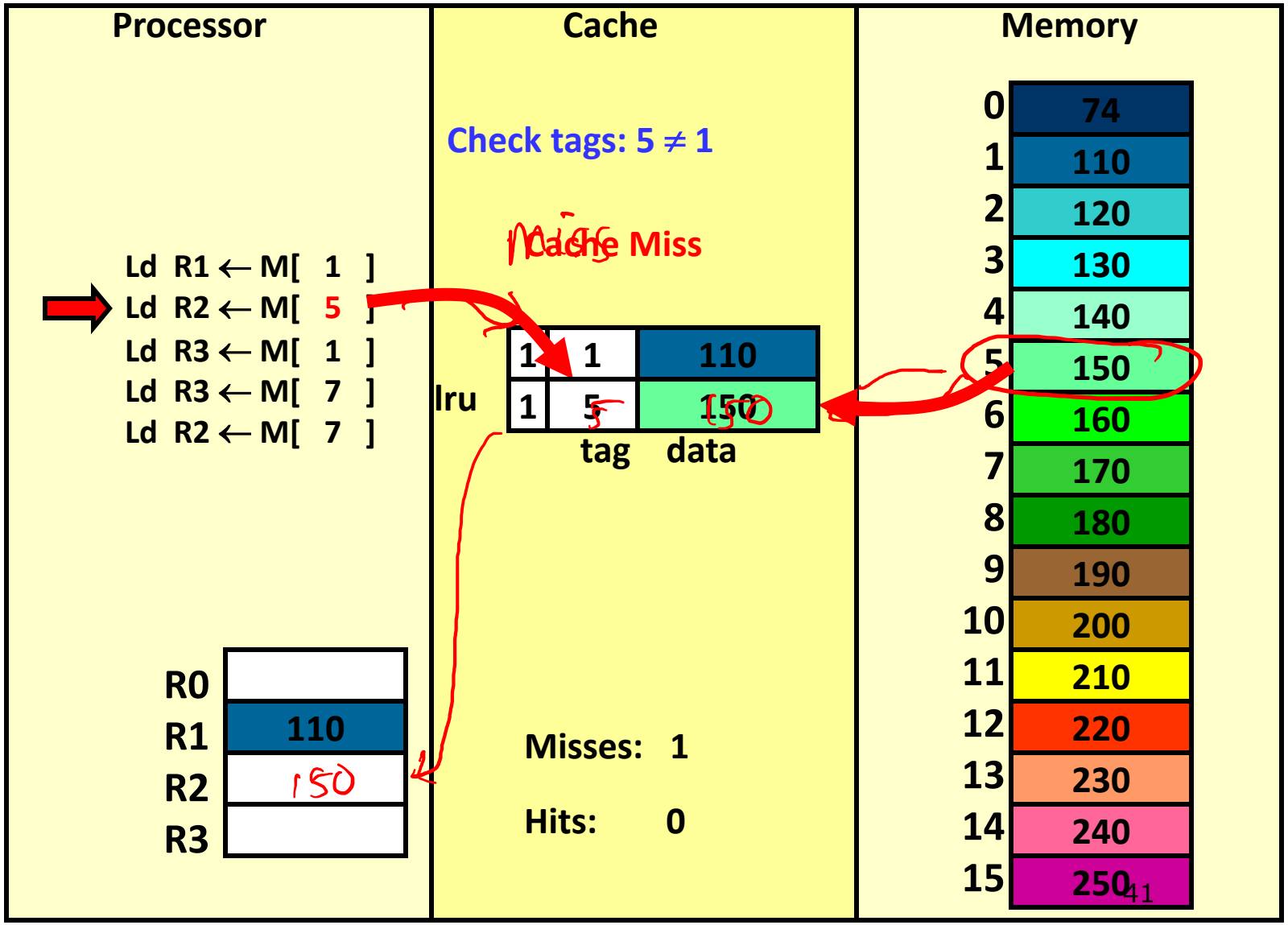
A Very Simple Memory System



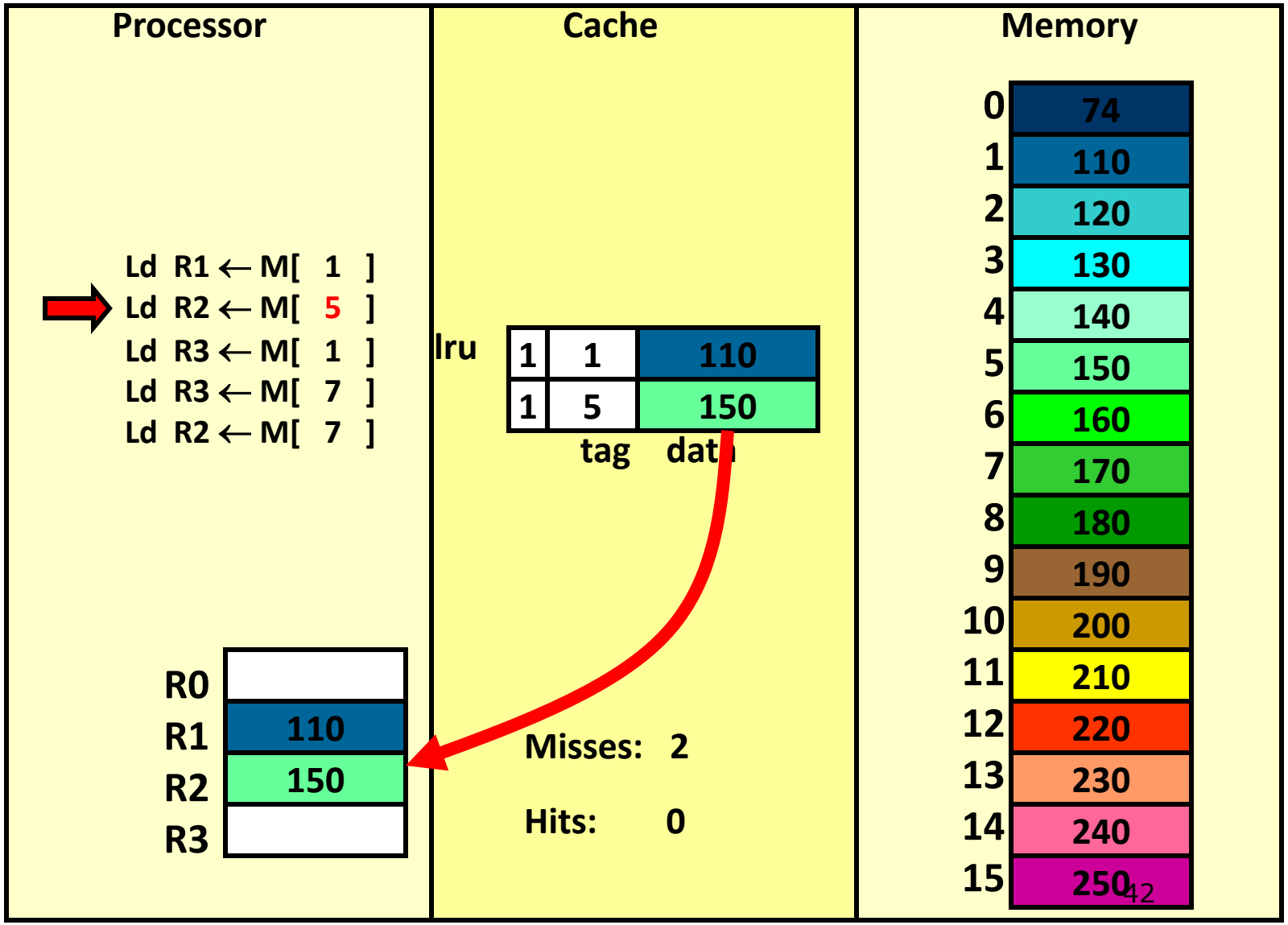
A Very Simple Memory System



A Very Simple Memory System



A Very Simple Memory System



A Very Simple Memory System

Processor

Ld R1 ← M[1]
Ld R2 ← M[5]
Ld R3 ← M[1]
Ld R3 ← M[7]
Ld R2 ← M[7]

R0	
R1	110
R2	150
R3	

Cache

Check tags: 1 ≠ 5,
but 1 = 1 (HIT!)

lru

	tag	data
1	1	110
1	5	150

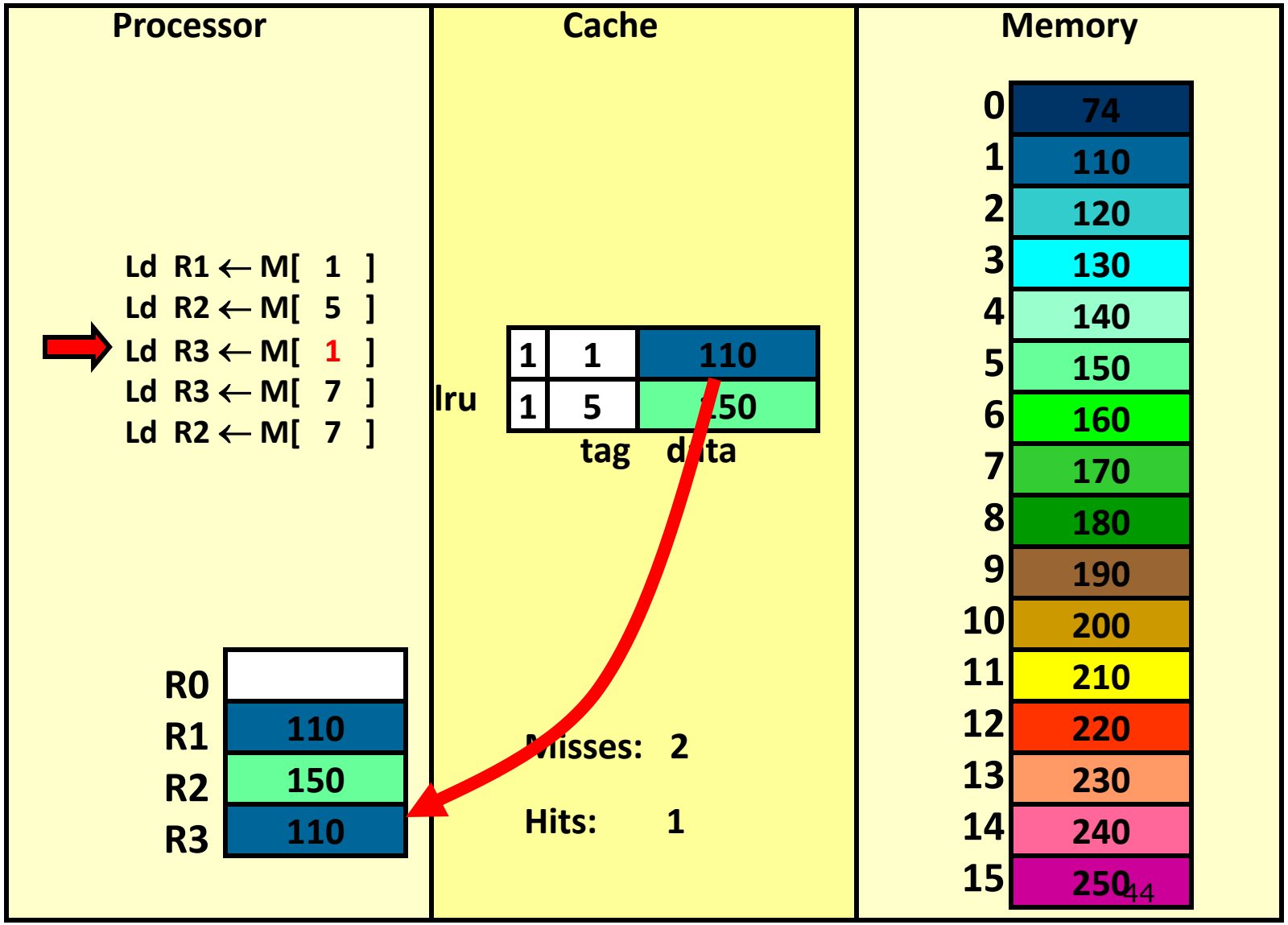
Misses: 2

Hits: 0

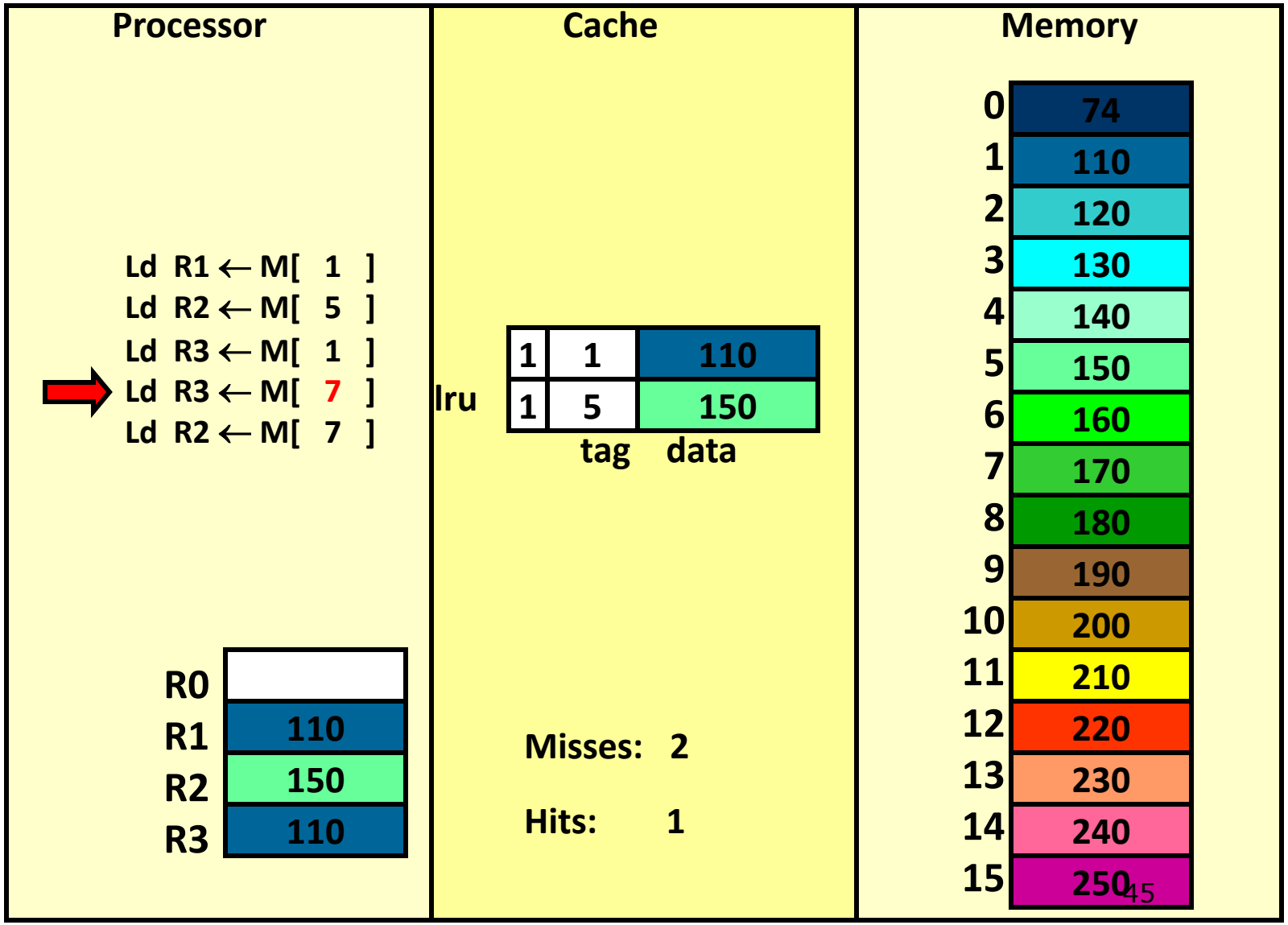
Memory

0	74
1	110
2	120
3	130
4	140
5	150
6	160
7	170
8	180
9	190
10	200
11	210
12	220
13	230
14	240
15	250

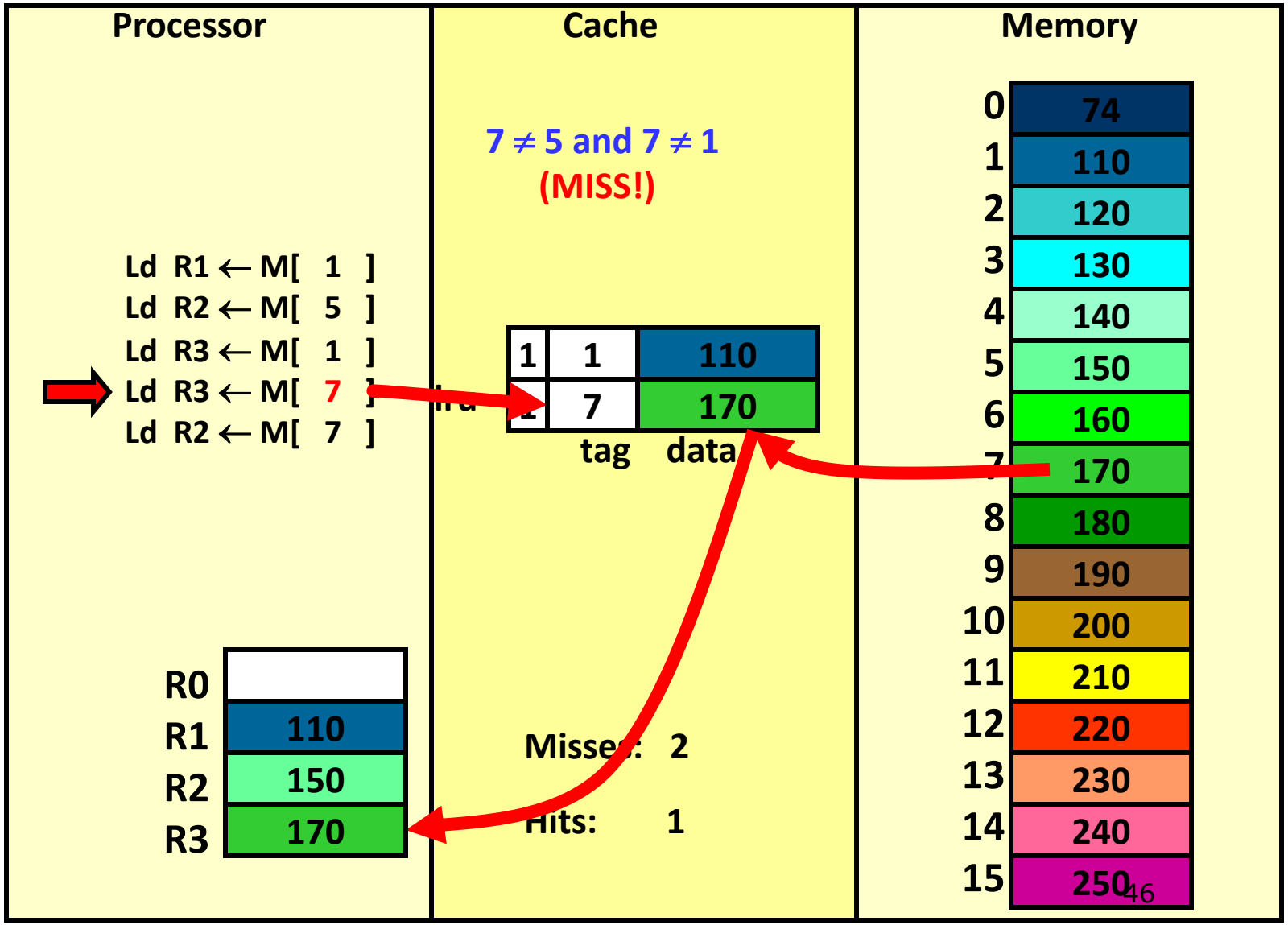
A Very Simple Memory System



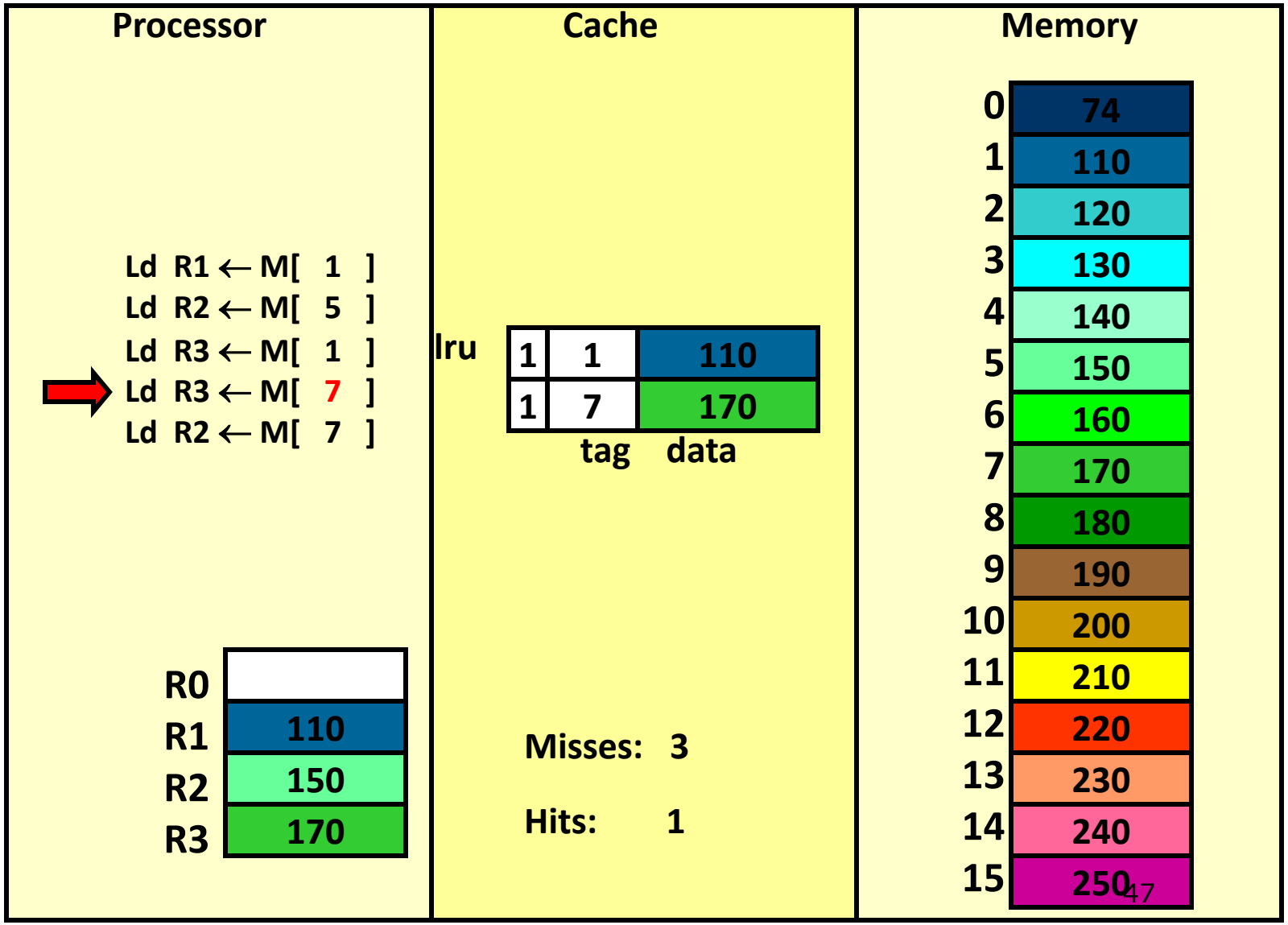
A Very Simple Memory System



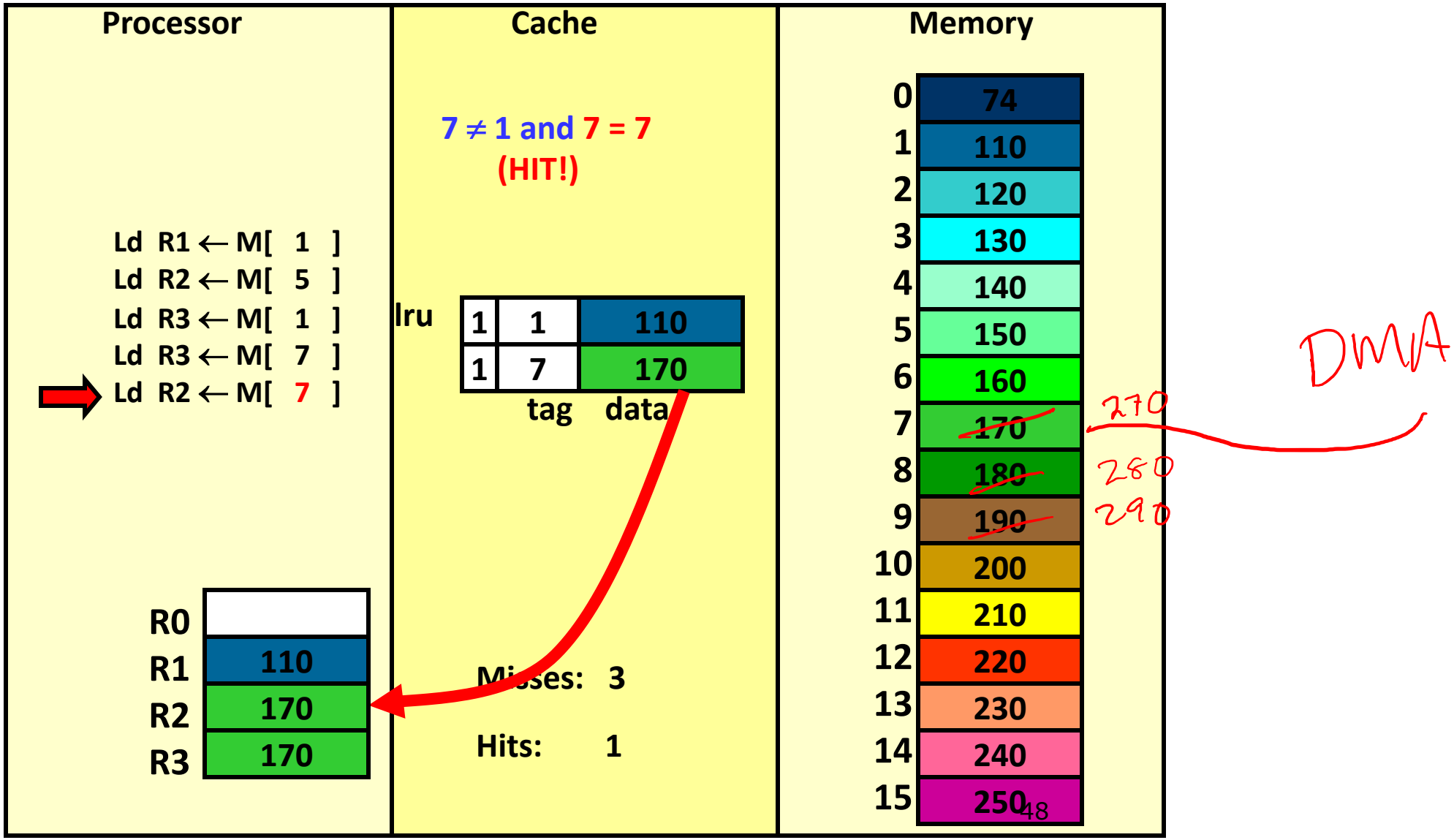
A Very Simple Memory System



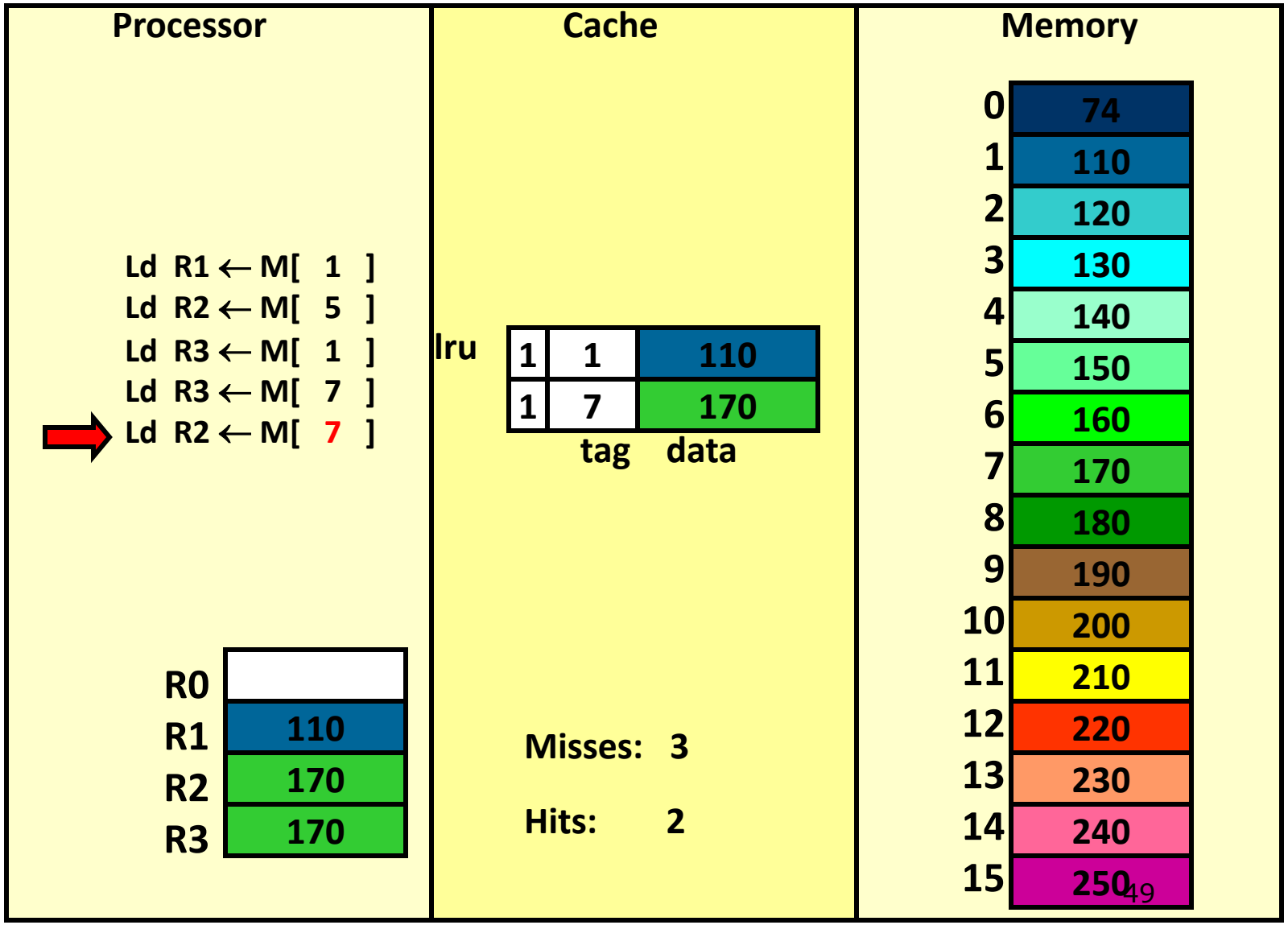
A Very Simple Memory System



A Very Simple Memory System

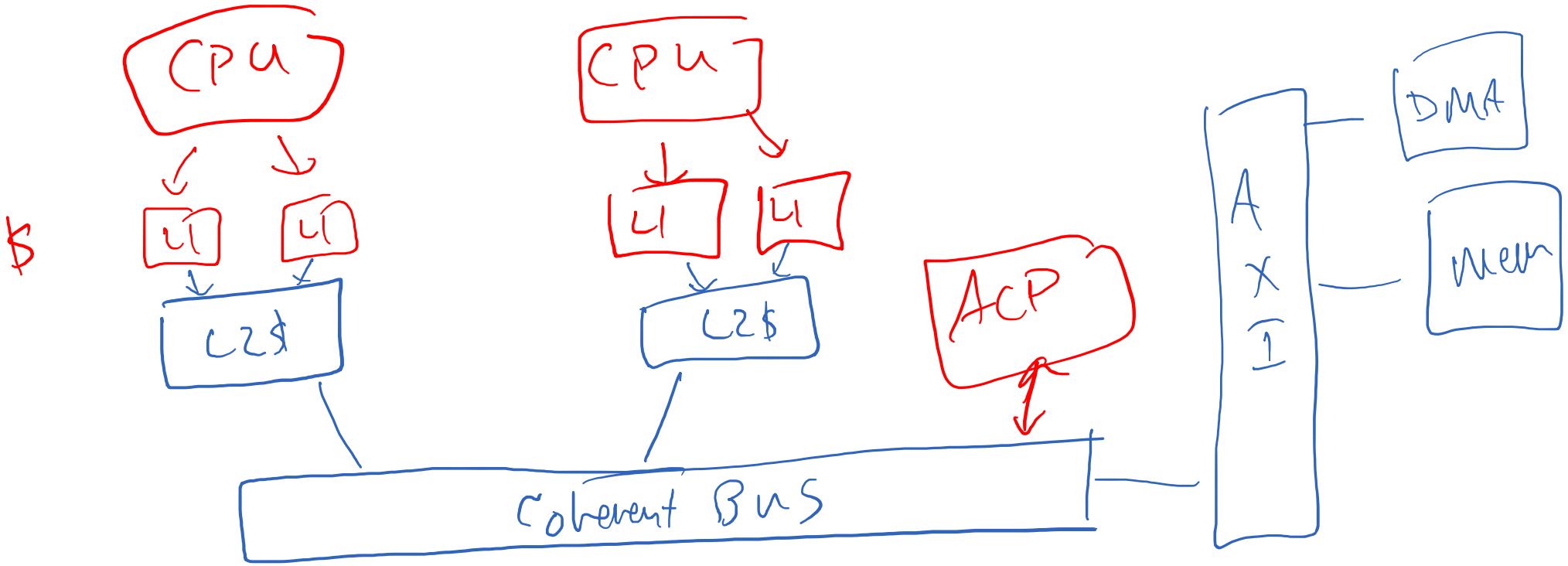


A Very Simple Memory System



Next Time:

- Caches (again)



10: Caches

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University